Spanner Feature Store

12 notebooks — SQL at the serving layer

Cloud Spanner | secondary indexes | ACID transactions | native vector search

github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/Spanner

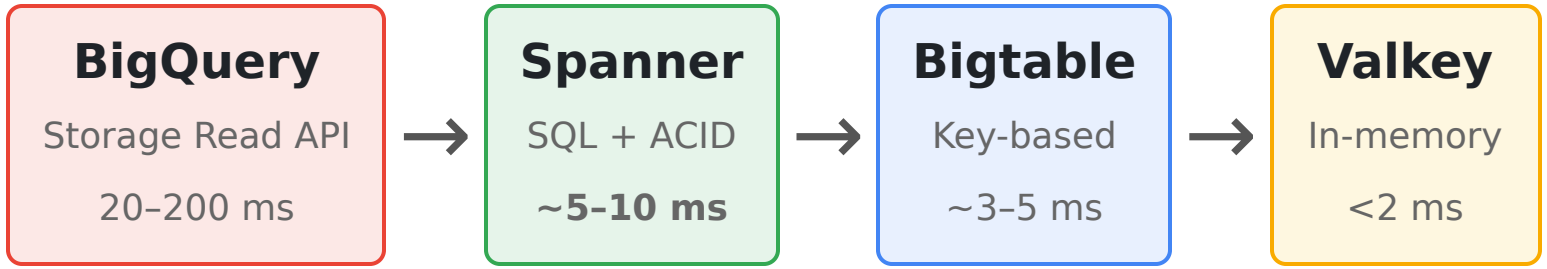
The Feature Store Problem

Features are computed in batch (BigQuery) but served in real time. The online store sits between them. **Spanner's angle: make the serving layer a real SQL database** — filter on any column, JOIN feature tables, update many rows atomically.

This series builds a complete feature store on **Cloud Spanner** — BigQuery as the offline engine, Spanner for SQL-powered online serving with strong consistency.

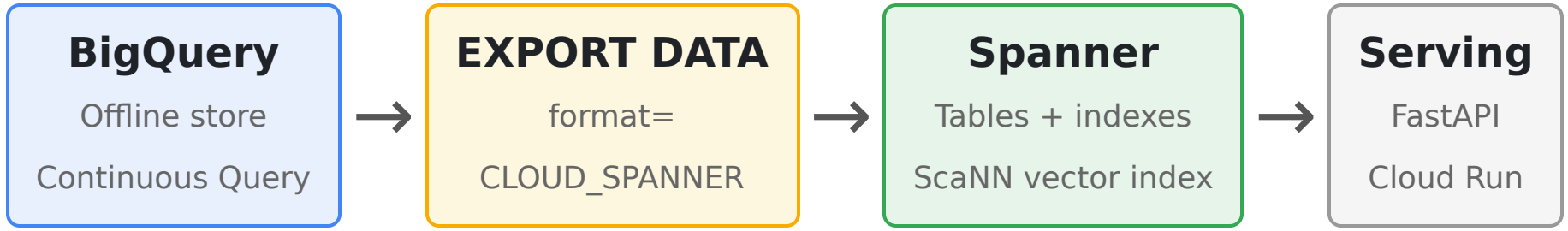
The Latency Spectrum

Where Spanner sits among the Google Cloud feature store approaches:



Spanner trades a few milliseconds versus key-value stores for **SQL at the serving layer**: secondary-index lookups on any column, serving-time JOINs, ACID transactions, and automatic multi-region with strong consistency (99.999% SLA).

Architecture



Features live in tables keyed by a **composite primary key** `(entity_group, entity_id)`. Change streams flow Spanner → BigQuery for the reverse path.

Concept	Spanner	Bigtable equivalent
Row key	composite PK `('A', '000001')`	concatenated `A#000001`
Non-key lookup	**secondary index**	not possible (key only)
Multi-table read	**serving-time JOIN**	pre-denormalized row
Multi-row write	**ACID transaction**	single-row atomic only

The Data

A **130K-entity dataset** shared across all feature store series — 26 entity groups (A–Z), 5,000 entities per group, 224 columns spanning every BigQuery data type.

Resource	Size	Purpose
`dense_features`	130,000 rows, 224 cols	Every BQ data type
`sparse_events`	2,600,000 rows	Time-series events
`spanner_feature_store`	BigQuery dataset	Independent per-series dataset

Native types map directly — `STRING`, `INT64`, `FLOAT64`, `BOOL`, `TIMESTAMP`, `NUMERIC`, `JSON`, `ARRAY` — **no serialization needed** (unlike Bigtable/Valkey, where everything is bytes). Loaded via mutations, batched to stay under the 80,000-mutation commit limit.

Fundamentals — SQL Reads & Query Plans

Read paths

- `execute_sql` — parameterized SQL
- `snapshot.read()` — key-based, skips SQL parse
- Strong vs stale snapshots

Measured (130K rows, 100 PU)

Operation	p50
Point read (strong)	~10 ms
Point read (`read()`)	~9 ms
Stale read (10s)	~8 ms

Reading the plan (`PROFILE` mode)

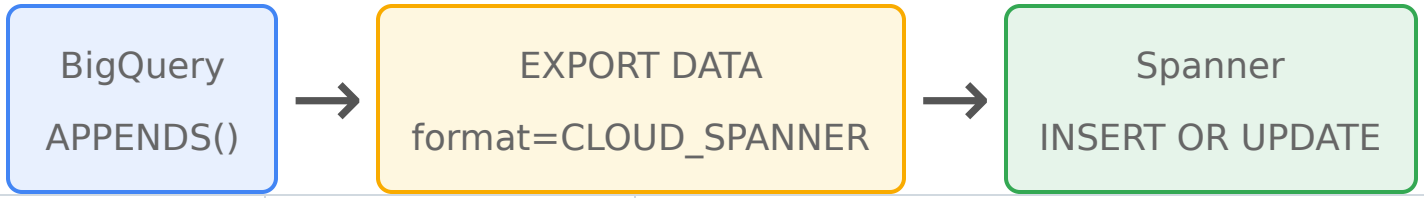
The table *is* its primary index — a PK lookup and a full-table filter both show as a `Scan`. The signal is `rows_scanned`:

Query	rows_scanned
PK point read	**1**
`WHERE feature_float_1 > 900`	**130,000**

That gap is what secondary indexes remove.

Synchronization — BigQuery ↔ Spanner

Spanner is a **native** `EXPORT DATA` **target**, just like Bigtable — the reverse-ETL path is one SQL statement, not a custom bridge.



Pattern	Freshness	Notes
Client `INSERT OR UPDATE`	Minutes–hours	Idempotent upsert, no custom dedup
One-time `EXPORT DATA`	Minutes	Needs BQ Enterprise reservation
Continuous query	Seconds	`EXPORT DATA` + `CONTINUOUS` reservation
Change streams → BigQuery	Seconds	Reverse sync via Dataflow template

`INSERT OR UPDATE` is a first-class idempotent primitive — re-applying a mutation yields the identical row. `change_timestamp_column` orders writes.

Secondary Indexes — Query by Any Column

Bigtable can only look up by row key. Spanner indexes any column at serving time — **its fundamental flexibility advantage**.

- `CREATE INDEX` on a feature column
- `STORING` — covering index, no base-table lookup
- `NULL_FILTERED` — skip NULLs (and the trade-off)
- Composite indexes — leading-column seek
- Interleaved (local) vs global indexes

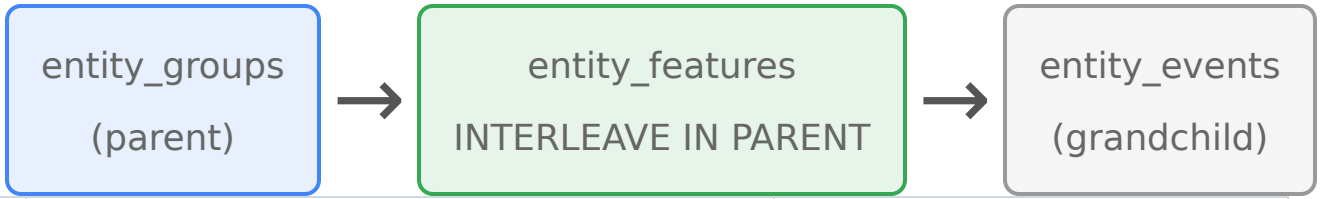
Query plan tells the story

- Basic index → Index Scan **+ Distributed Cross Apply** (back-join to base)
- Covering index (`STORING`) → **Index Scan only**

Write amplification (measured): each secondary index adds index-entry mutations to every write — ~13% slower at 7 indexes vs 0.

Interleaved Tables — Co-Location

Parent-child tables stored on the **same split** — a guarantee, not a hint.



Concept	Interleaving	Foreign keys
Co-location	**Yes** — same split	No — rows anywhere
Cascade delete	`ON DELETE CASCADE` (1 mutation)	Manual
Referential integrity	Yes	Yes

A single-row parent `DELETE` cascades to all children and grandchildren as **one logical operation**. Co-location keeps JOINs local — at scale the flat-table JOIN needs a cross-split operator the interleaved one avoids.

Transactions — ACID at the Serving Layer

Spanner's defining property: multi-row, multi-table, serializable transactions.

Primitives

- `run_in_transaction` — read-modify-write
- Automatic **abort + retry** on conflict
- Batch DML, partitioned DML
- Stale reads (`exact_staleness`, `read_timestamp`)
- Commit timestamps (monotonic, TrueTime)

Concurrency, measured

10 threads incrementing one row:

- Transactional → **final = 10** (31 retries, 0 lost)
- Naive read-then-write → **lost 9 updates**

Lock contention on a hot key: p50 23 ms → **290 ms** under 6 writers.

Schema Evolution — Online DDL

Add, drop, and reshape columns with no downtime — proven, not asserted.

- `ADD COLUMN` — non-blocking; reads/writes continue
- `DEFAULT` values — backfill-free
- **Generated columns** (`AS (...) STORED`)
- Type change: add → backfill → drop
- `INFORMATION_SCHEMA` runtime discovery

Non-blocking proof (measured)

While `ADD COLUMN` ran (~5 s), concurrent traffic kept flowing: **129 reads + 135 writes succeeded, 0 errors.**

A discovery-driven `SELECT` list tolerates not-yet-added columns — forward-compatible serving.

Serving Integration

Train on BigQuery, serve from Spanner with JOINs at request time.

Serving path

- `read()` vs `execute_sql` vs reused snapshot
- Serving-time 1/2/3-table JOINs
- Session pool sized to concurrency
- Hardened client: timeout, miss, stale fallback
- FastAPI endpoint

Measured (100 PU)

Method	p50
`read()` reused snapshot	~8.8 ms
`read()` key-based	~9.9 ms
`execute_sql` (params)	~10.7 ms

A pool smaller than concurrency serializes requests → p99 balloons.

Vector Search — Native KNN & ANN

Spanner has **native** vector search — exact KNN scalar functions and ScaNN approximate indexes. Its unique angle: combine a SQL filter and vector distance in **one statement**.

Exact KNN

`COSINE_DISTANCE` / `EUCLIDEAN_DISTANCE` / `DOT_PRODUCT`

`ORDER BY ... LIMIT k`

ANN (ScaNN)

`CREATE VECTOR INDEX` + `APPROX_COSINE_DISTANCE`

tune `num_leaves_to_search`

The differentiator

```
SELECT item_id,
  COSINE_DISTANCE(embedding, @q) d
FROM item_catalog
WHERE category='electronics'
  AND price < 500
ORDER BY d LIMIT 10
```

Metadata filter + KNN, server-side, one round-trip.

Vector Search — Across the Series

All four serving stores now have **native** vector search — they differ on index, pre-filtering, and latency.

Approach	Index	Pre-filtering	Notes
Spanner	ScaNN ANN + exact	**Full SQL WHERE**	filter + KNN in one query
Valkey	HNSW / FLAT	TAG + NUMERIC	sub-5 ms, in-memory
BigQuery	IVF / TreeAH	SQL pushdown	offline scale
Bigtable	brute-force	row key only	no ANN

Spanner when vector search is one step inside a richer SQL query — filter many columns, JOIN, then rank by distance. **Valkey** for sub-ms standalone retrieval.

Multi-Region & High Availability

Regional gives 99.99%; multi-region gives **99.999%** with automatic, strongly-consistent failover.

`nam3` config

- N. Virginia `us-east4` — R/W (leader)
- S. Carolina `us-east1` — R/W
- Iowa `us-central1` — witness

Strong reads hit the leader quorum; stale reads serve from a local replica.

Routing & consistency

- `DirectedReadOptions` — pin reads to a region (the app-profile analog)
- Witness replicas vote, serve no reads
- Read-your-write: zero replication lag (proven on the regional instance)

99.99% ≈ 52 min/yr down; 99.999%

Local Development — Emulator

The Spanner emulator runs the same client code locally, free, for CI.

- `gcloud emulators spanner start`
- `SPANNER_EMULATOR_HOST` — client auto-routes
- pytest fixtures + GitHub Actions / Cloud Build
- Readiness poll instead of fixed sleep

Can test: transactions, JOINs, secondary indexes, DDL, **and native vector search** (COSINE_DISTANCE, vector index, APPROX_* all supported).

Can't: multi-region, fine-grained IAM, real latency.

A 30-operation suite runs in well under a second — fast enough for every push.

Capstone — Recommendation Engine

A multi-table recommender exercising every Spanner capability, with native vector search in the pipeline.

- **Stage 1** — SQL filter **+** vector KNN in one statement (~45 ms)
- **Stage 2** — serving-time JOINs (user × items × interactions)
- **Stage 3** — score & rank
- **Stage 4** — atomic cross-table update (impossible in Bigtable)

- 3 tables: `user_profiles`, `item_catalog`, `user_item_interactions`
- ScaNN vector index for ANN candidate retrieval
- FastAPI `/recommend` + concurrent load test (p50/p95/p99, QPS)
- Terraform: autoscaling, PITR, backup schedule, least-privilege IAM

Production Operations

Deploy — Terraform (`google_spanner_instance` with `autoscaling_config`, `google_spanner_database` with DDL + PITR + backup schedule)

Monitor — Cloud Monitoring: CPU utilization (alert at 65% regional), request latencies; `SPANNER_SYS.QUERY_STATS_TOP_*`

Secure — least-privilege IAM: `databaseReader` (serving), `databaseUser` (sync), `databaseAdmin`

Scale — Processing Units (1000 PU = 1 node) + autoscaling; Enterprise edition for vector indexes

HA — regional: 3 R/W replicas, auto failover, 99.99%; multi-region for 99.999%

Readiness — deletion protection, autoscaling bounds, backups, alerting, IAM split, reproducible from Terraform

When to Use Spanner

Choose Spanner when the serving layer needs to be a **real SQL database** — filter/JOIN on non-key columns, multi-row/multi-table **ACID** updates, vector search *inside* a richer SQL query, or automatic **multi-region** strong consistency (99.999%).

Capability	Spanner	Bigtable	Valkey
Point-lookup latency	~5–10 ms	~3–5 ms	<2 ms
Query by any column	**secondary index**	row key only	secondary index
Serving-time JOINs	**yes**	no (denormalize)	no
Multi-row transactions	**ACID**	single-row	single-key
Vector search	ScaNN (native)	brute-force	HNSW (native)
BQ sync	EXPORT DATA + change streams	EXPORT DATA	Pub/Sub bridge

Resources

Repository: github.com/statmike/vertex-ai-mlops/MLOps/Feature Store/Spanner

Cloud Spanner: cloud.google.com/spanner/docs/overview

Vector search: cloud.google.com/spanner/docs/find-k-nearest-neighbors

Secondary indexes: cloud.google.com/spanner/docs/secondary-indexes

12 notebooks · Cloud Spanner · SQL serving · ACID · native vector search